Task 1: Create the foodieAppTester IAM user

Procedure:
1. Open AWS IAM → Users → Create user.
2. Enter foodieAppTester.
3. Create the user without granting broad permissions.
4. Create a customer-managed policy named StorageBucketListOnly.
5. Attach the policy to the user.

Policy JSON:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListStorageBuckets",
      "Effect": "Allow",
      "Action": "s3:ListAllMyBuckets",
      "Resource": "*"
    }
  ]
}
```

Task 2: Audit an existing IAM user or role

Example audit for a food delivery application backend:

Permission found	Why it may be too broad
s3:* on all resources	Allows unnecessary operations across S3 resources.
ec2:* on all resources	Allows management actions unrelated to order processing.

dynamodb:* on all resources	Grants more database permissions than a specific application task may need.

Task 3: Modify the overly broad permissions

For example, a backend that only needs to read and update order records and read food-menu data could use a more restricted policy.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadFoodMenu",
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:Query"
      ],
      "Resource":
"arn:aws:dynamodb:ap-south-1:123456789012:table/FoodMenu"
    },
    {
      "Sid": "ReadAndUpdateOrders",
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:Query",
        "dynamodb:UpdateItem"
      ],
      "Resource":
"arn:aws:dynamodb:ap-south-1:123456789012:table/Orders"
    },
    {
      "Sid": "ReadOrderObjects",
```

```
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::foodie-order-storage/*"
    },
    {
      "Sid": "ListOrderStorage",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::foodie-order-storage"
    }
  ]
}
```

Explanation:
- The policy limits database actions to the FoodMenu and Orders tables.
- It permits reading and updating orders without granting order deletion.
- It limits S3 access to reading objects in the specified storage bucket.
- It does not grant general EC2 administration or unrestricted S3 access.

Task 4: Upload profile pictures only

Policy name: UploadProfilePicturesOnly

The following policy allows a test user to upload files to the user-profile-pics bucket.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "UploadProfilePictures",
      "Effect": "Allow",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::user-profile-pics/*"
```

```
    }
   ]
}
```

Task 5: Final presentation or one-page report

Implementing Least-Privilege Access in AWS IAM

Objective

To improve cloud security by creating IAM identities, assigning specific permissions, enabling MFA, and applying the principle of least privilege.

Configurations Implemented

- Created the InstaUploader IAM user and the ContentCreators group.
- Created read-only S3 permissions for the playlistViewer user.
- Created an inline policy to start and stop EC2 instances tagged Environment=Dev.
- Created a managed policy for viewing DynamoDB table metadata.
- Configured MFA and reviewed secure authentication practices.
- Created restricted policies for storage access, order processing, and profile-picture uploads.
- Before-and-After Comparison
- Before: Broad permissions such as s3:*, ec2:*, or Action: "*" can give users unnecessary access to cloud resources.

After: Permissions are restricted to required actions, specific resources, and appropriate access conditions. For example, an upload-only user can upload profile pictures without receiving general bucket administration permissions.

Security Improvements

1. Enforce MFA for privileged identities.
2. Prefer temporary credentials and IAM roles for applications.

3. Review unused permissions and access logs regularly.
4. Use resource-specific policies and test changes before deployment.
5. Monitor suspicious activity using CloudTrail and appropriate alerting.

Conclusion

AWS IAM provides centralized control over cloud identities and permissions. Applying least privilege, MFA, regular audits, and monitoring helps reduce unauthorized access and improves the security of cloud applications such as food delivery and e-commerce platforms.